



Posible Solución T5 formativo Teorico

Contexto

En el taller 5 formativo se resolvió un ejercicio de backtracking, donde queríamos resolver si existía un camino de exactamente K pasos en un laberinto con obstáculos desde la esquina superior izquierda a la esquina superior derecha. La pregunta teórica de este taller, pedía una posible mejora al algoritmo de backtracking realizado. Existen muchas posibles respuestas a esta pregunta y por eso bastaba con que entregaran una sola. Se proponen algunas como guía:

Podas

Las podas consisten en agregar mas condiciones al problema que no están en el enunciado original pero que puedan inferirse de este, con el objetivo de "podar" ramas del árbol de recursión y hacer el algoritmo más rapido. Esto se traduce en agregar condiciones a la función de Check

- Poda Manhattan: La distancia Manhatta es la distancia mínima entre un punto de la grilla y otro, se calcula solo restando las coordenadas de ambos puntos:

$$\text{dist_manhattan} = \text{abs}(i - \text{goal_i}) + \text{abs}(j - \text{goal_j})$$

Como debemos llegar al objetivo en exactamente K pasos, si estamos en una casilla (i, j) y nos quedan M pasos para dar, entonces podemos calcular la distancia Manhattan y si es que es mayor a la cantidad de pasos que nos quedan, es imposible llegar al objetivo, nos faltarán pasos para llegar.

Por lo tanto agregamos a la función de check lo siguiente:

```
dint dist = abs(i - M-1) + abs(j - M-1) #la casilla objetivo es (M-1, M-1);  
if (K < dist) return false;
```

- Poda por celda encerrada: Si llegamos a una celda donde todos sus vecinos son inválidos (son todos obstáculos o la casilla de la que venimos) entonces podemos decir que a pesar de que esa casilla sea valida para pisar, no llegaremos a ningún lado ya que estamos encerrados. Esto se ve así en el check:

```
if ((matrix[i-1][j] == 1 || matrix[i-1][j] == 2) &&  
    (matrix[i+1][j] == 1 || matrix[i+1][j] == 2) &&  
    (matrix[i][j-1] == 1 || matrix[i][j-1] == 2) &&  
    (matrix[i][j+1] == 1 || matrix[i][j+1] == 2)) return false;
```

Heurísticas

Las heurísticas lo que hacen es ordenar los valores del dominio de tal forma que al explorar esas asignaciones primero, es mas probable que lleguemos a una solución antes (en el peor caso, igual las probaremos todas).

- Heurística de ordenar pasos: Como sabemos que partimos en la casilla $(0, 0)$ y el objetivo es la $(M - 1, M - 1)$ el camino directo siempre es una diagonal de abajo hacia arriba y de izquierda a derecha. Por lo que, si ordenamos los movimientos en la grilla de tal forma que nos movamos primero a la derecha, después hacia debajo y después los demás movimientos, es mas probable que lleguemos a una solución antes (recordar que esto no asegura una solución, es posible que el camino del laberinto implique subir o ir a la izquierda, solo es mas probable que lleguemos antes si nos movemos primero a la derecha o izquierda).

En código, solo basta con ordenar los if que representan los movimientos de la forma explicada.

Propagación

Una propagación es cuando transmitimos información que fue obtenida por una rama del backtracking que puede servir en la exploración de otra rama distinta. Así, propagamos valores para hacer el backtracking mas inteligente.

- Propagación de casillas imposibles: Si en nuestra ejecución del backtracking llegamos a una casilla (i, j) faltando exactamente M pasos, luego de los llamados recursivos del backtracking no logramos llegar a una solución, podemos marcar esa casilla como invalida con M pasos, de esta forma, cualquier otro camino que llegue a esa casilla y que tenga igual o menos pasos restantes que M sabrá inmediatamente que no es posible llegar a una solución desde esa casilla. De esta forma nos evitamos tener que explorar muchas veces múltiples caminos que partes en una casilla, solo almacenamos esa información la primera vez que lleguemos a ella y propagamos esa información.

Para implementarlo es un poco mas complejo, lo que podemos hacer que ahora las casillas del tablero no solo sean números, sino que sean structs que almacenen lo siguiente:

```
struct cell {
    int value;
    int min_invalid_steps;
    bool reached;
}
```

Al iniciar, le asignamos a cada struct cell el valor de esa casilla y ponemos `min_invalid_steps` en $K + 1$ (esto es solo un ejemplo de un numero que siempre será mas grande que mi cantidad de pasos) y `reached` en false para denotar que todavía no se ha explorado esta casilla.

Luego en nuestra ejecución, del backtracking, debemos de ajustarlo para ahora usar las structs en vez de solo los valores, pero lo mas importante es que después pasar todos los if de los movimientos, si es que ninguno de ellos retorno true y ademas la cantidad de pasos que me queda es menor que el `min_invalid_steps` de la casilla lo actualizamos con mi la cantidad de pasos que me quedan y marcamos `reached` como true.

Luego, podemos agregar una condición en el check, que si la cantidad de pasos que me queda, es menor que `min_invalid_steps` de la casilla y ademas `reached` es true, entonces retornamos false porque ese camino ya lo exploramos y sabemos que no llega a una solución.

Es importante que se maneje la lógica del `min_invalid_steps` ya que como funciona el backtracking, podría ocurrir que la primera vez que se llegue a una casilla, sea con un camino muy largo, existiendo uno mas corto, por lo que la propagación solo es valida, para caminos que sean mas largos que `min_invalid_steps`.